

Capítulo 8

Modularidad

8.1 Dividir para vencer

Con las estructuras vistas hasta ahora del *PASCAL* se podrían construir programas que realizaran tareas muy complejas pero con escasa legibilidad. Un modo muy común, y generalmente muy efectivo, de resolver un problema es desmenuzarlo en una serie de problemas más sencillos y resolver después cada uno de ellos. Es la estrategia que ha recibido el nombre *dividir y vencer* o diseño *Top-Down*. Desde el punto de vista de la legibilidad es deseable que un programa de ordenador pueda estructurarse siguiendo lo más posible el esquema conceptual que se ha pensado utilizar para resolver el problema. Por ejemplo, supongamos que se trata de escribir un programa que calcule la suma de los n primeros términos de una sucesión geométrica de razón r y cuyo primer término es a . Para ello pensamos utilizar la expresión:

$$\sum_{i=0}^{n-1} ar^i = \frac{a(1 - r^n)}{1 - r}$$

y el esquema que podemos trazar para el programa es el siguiente:

Suma de la progresión:

- (1) Lectura de datos
- (2) Cálculo de la expresión
- (3) Salida de resultados

A su vez, el cálculo de la expresión matemática no es inmediato puesto que es necesario calcular r^n donde r es real y n entero. También podemos descomponer la tarea (2) en:

- (2) Cálculo de la expresión
 - (2.1) Obtener r^n
 - (2.2) Realizar multiplicaciones y divisiones.

A través de la descomposición modular que permite el *PASCAL* con las construcciones **Procedure** y **Function** es posible escribir el programa de un modo muy similar al pseudo código anterior. El cuerpo principal del programa será simplemente:

```

Program SumaProgresion;
Var
  n : Integer;
  r,a,suma : Real;
  (* Declaracion de procedimientos y funciones *)
  . . . . .
  (* Fin de la parte declarativa *)
Begin { SumaProgesion }
  { Se utiliza la expresion:   suma = a (1-r**n) / (1-r) }

  LeerDatos(n,a,r);

  CalcularExpresion;

  SacarResultados(suma);

End. { SumaProgresion}

```

y en el espacio reservado para declaraciones en el programa se especificará qué tareas se han de realizar en los procedimientos **LeerDatos**, **CalcularExpresion**, y **SacarResultados**, definidos por el programador.

8.2 Procedimientos

La sintaxis con la que se especifican estos procedimientos es similar a la de un programa *PASCAL* pero se usa la palabra reservada **Procedure** para indicar que se trata de un procedimiento parcial:

Procedure *nombre* (argumentos) ;

```
    Parte declarativa
Begin
    Bloque de sentencias del procedimiento
End;
```

La parte declarativa del procedimiento tiene el mismo fin que en un programa y en ella se definen tipos de datos, variables y constantes como en un programa y también, si así se requiere, procedimientos y funciones que sean necesarios para la realización de este **Procedure**. En el bloque principal del procedimiento se escriben las sentencias *PASCAL* que constituyen el algoritmo que ha de realizar, y en él se pueden utilizar datos definidos en su parte declarativa, en la parte declarativa del programa principal o bien que aparecen como argumentos del procedimiento. Estos argumentos se utilizan para comunicar valores de datos al procedimiento desde el bloque donde ha sido llamado. Cuando se llama al procedimiento, estos argumentos van separados por comas. En la declaración del procedimiento los argumentos se especifican de un modo muy parecido a como se especifica el tipo de datos en la declaración de las variables de un programa. Los datos de un mismo tipo van separados por coma y se utiliza el punto y coma para separar datos de distinto tipo. A continuación de los argumentos se especifica el tipo de variable. Por ejemplo, al procedimiento **SacarResultados** se le llama con el argumento **suma** que es un dato **Real**. Este procedimiento se puede escribir así:

```
Procedure SacarResultados(resultado : Real);
Begin { SacarResultados }
    Writeln('');
    Writeln('La suma de la series es: ', resultado:10:6);
End; { SacarResultados }
```

En el argumento se especifica el nombre que se va a utilizar en el procedimiento y también de qué tipo de dato se trata. El nombre no ha de coincidir con el utilizado en el bloque original, puesto que la identificación de datos la realiza el compilador por el orden en el que aparecen en el argumento y no por el nombre.

8.3 Funciones

Muy parecida a la estructura **Procedure** es la estructura **Function** y se utiliza para aquellos casos en los que se desea que el dato que se calcula en el módulo independiente pueda aparecer en cualquier instrucción *PASCAL* en el mismo lugar en el que aparecería el dato. Este es el caso de las funciones intrínsecas del *PASCAL* que ya se han visto. La sintaxis de la estructura **Function** es:

```

Function nombre ( argumentos ) : Tipodato ;
    Parte declarativa
Begin
    Bloque de sentencias de la función
End;

```

y en el bloque se especifica el valor que debe devolver la función asignándoselo al *nombre* utilizado para nombrar la función. Por ejemplo, en el procedimiento **CalcularExpresion** debemos incluir el cálculo de la potencia de un número real a un exponente entero. Esta operación, que puede aparecer en muchos otros programas, merece la pena que se escriba en un módulo aparte, y dado que el resultado es un dato que suele aparecer en expresiones aritméticas, es razonable que se escriba como **Function**. A la función le llamamos **Poten**:

```

{Funcion que eleva un numero real a un potencia entera }
Function Poten ( base : Real;
                exponente : Integer ) : Real;

Var
    i : Integer;
    z : Real;
Begin { Poten }
{1}   If exponente <= 0 Then
{2}       Poten :=1
{3}       Else
{4}       If exponente = 1 Then
{5}       Poten := base
{6}       Else
{7}       Begin
{8}       z := base;

```

```

{9}          For i := 2 to exponente Do
{10}                      z := z * base;
{11}                      Poten := z
{12}          End;
End; { Poten }

```

En el argumento se especifican las dos variables con las que nos vamos a referir al exponente y a la base, el exponente ha de ser un valor entero y la base puede ser real. A continuación, se especifica que el valor que ha de devolver la función al bloque donde ha sido llamada es **Real**. Para realizar el cálculo se necesitan otras dos variables y se definen en la parte declarativa. En las instrucciones {2}, {5} y {11}, aparece el nombre de la función en la parte izquierda de una asignación y es en ellas donde se especifica el valor que puede devolver la función. La llamada a **Poten** desde el procedimiento **CalcularExpresión** :

```

{Calculo de la expresion para la suma de la serie }
Procedure CalcularExpresion ;
Begin { CalcularExpresion }
    suma := a * ( 1.0 - Poten(r,n) ) / ( 1.0 - r) ;

End; { CalcularExpresion }

```

se realiza dentro de una expresión aritmética.

8.4 Ámbito de definición de las variables

Las variables que se definen en la parte declarativa de un procedimiento o una función existen solamente para ese módulo. Se reservan lugares de memoria para acomodarlas cuando se requiere el cálculo del módulo y se liberan cuando se termina la ejecución de los algoritmos especificados en el módulo. Por tanto, si en algún otro módulo del programa, o en la parte principal, se utilizan esas variables, el compilador avisará de que no se conoce el tipo de dato de esos identificadores. Por otro lado, se pueden utilizar los mismos nombres para variables definidas en distintos módulos ya que el compilador interpretará correctamente el hecho de que con cada nombre se refiere al lugar de memoria reservado para el dato que se va a manipular en cada módulo. El único modo que hay para pasar valores de datos desde un

módulo a otro, es a través de los argumentos del **Procedure** o **Function**. Sin embargo, desde el bloque principal se pueden pasar datos a los procedimientos o funciones : cualquier variable o constante definido en la parte declarativa del bloque principal es accesible a todos los módulos declarados para el programa (tanto para conocer su valor como para modificarlo). Por eso, el **Procedure CalcularExpresion** sin tener ningún argumento puede acceder al valor de los datos almacenados en las variables **n**, **a** y **r**. Con esta posibilidad parece lógico preguntarse el porqué de utilizar argumentos en los módulos. En particular, el porqué de los argumentos en la función **Poten**. Pues bien, la ventaja de utilizar los argumentos **base** y **exponente** es que, aparte de mejorar la legibilidad, el mismo código puede utilizarse en otros programas independientemente de cómo se llamen las variables que intervienen en el cálculo.

8.5 Paso de valores por contenido o dirección

Lo dicho anteriormente sobre las variables definidas en las partes declarativas de los módulos, también es cierto para los argumentos: se reservan para esos datos nuevos lugares de memoria cada vez que se llama a los módulos, se copia a ellos los datos almacenados en los argumentos, se utilizan en el procedimiento y luego se liberan. Por tanto, dentro de un módulo no se puede alterar el valor de las variables definidas para el bloque que llama al módulo. Cuando se desee modificar en el módulo el valor de una variable pasada como argumento hay que especificarlo y así será accesible al módulo no sólo el contenido de la variable sino también su dirección en la memoria. La especificación se realiza anteponiendo a estas variables la palabra reservada **Var**. Por ejemplo, en el procedimiento **Leerdatos**

```

Procedure Leerdatos( Var numero: Integer;
                    Var primero , razon : Real );
Begin {Leerdatos}
    Writeln ( ' Primer termino de la serie? ');
    Readln (primero);
    Writeln ( ' numero de terminos? ');
    Readln (numero);
    Writeln ( ' Razon de la serie? ');
    Readln (razon);
End; {Leerdatos}

```

se especifica que las variables `numero`, `primero`, y `razon`, son argumentos *pasados* al módulo por *referencia* o *dirección* y no por *contenido* o *valor*. Los datos que almacena el procedimiento en `numero`, `primero`, y `razon`, en realidad se están almacenando en los lugares de memoria que el programa principal reservó para `n`, `a`, y `r`.

El programa completo que realiza la suma de la progresión es:

```
Program SumaProgresion;
Var
  n : Integer;
  r,a,suma : Real;

  { Lectura de datos }
  Procedure Leerdatos( Var numero: Integer;
                      Var primero , razon : Real );
  Begin {Leerdatos}
    Writeln ( ' Primer termino de la serie? ');
    Readln (primero);
    Writeln ( ' numero de terminos? ');
    Readln (numero);
    Writeln ( ' Razon de la serie? ');
    Readln (razon);
  End; {Leerdatos}

  {Funcion que eleva un numero real a un potencia entera }
  Function Poten ( base : Real;
                  exponente : Integer ) : Real;
  Var
    i : Integer;
    z : Real;
  Begin { Poten }
    If exponente <= 0 Then
      Poten :=1
    Else
      If exponente = 1 Then
        Poten := base
      Else
        Begin
```

```

        z := base;
        For i := 2 to exponente Do
            z := z * base;
        Poten := z
    End;
End; { Poten }

{Calculo de la expresion para la suma de la serie }
Procedure CalcularExpresion ;
Begin { CalcularExpresion }
    suma := a * ( 1.0 - Poten(r,n) ) / ( 1.0 - r ) ;

End; { CalcularExpresion }

Procedure SacarResultados(resultado : Real);
Begin { SacarResultados }
    Writeln('');
    Writeln('La suma de la series es: ', resultado:10:6);
End; { SacarResultados }

Begin { SumaProgesion }
    { Se utiliza la expresion:  suma = a (1-r**n) / (1-r) }

    LeerDatos(n,a,r);

    CalcularExpresion;

    SacarResultados(suma);

End. { SumaProgresion}

```

El ámbito de las variables y la diferencia entre el paso de argumentos por referencia o contenido se puede apreciar bien con el siguiente programa **Uno**:

```

Program Uno;
Var
    a , b ,c : Integer;

```

```

        Procedure Escribe ( texto  : String;
                           Var x   : Integer;
                           y       : Integer );

        Begin { Escribe }
            x := 2 * x ;
            y := 2 * y ;
            Writeln (texto, x:4 , y:4 , c:4 );
        End; { Escribe }

    Begin
{1}      a := 3;   b := 5;   c:= 7;
{2}      Writeln(' Antes : ', a:4 , b:4 , c:4 );
{3}      Escribe('Durante: ', a , b );
{4}      Escribe('Durante: ', a , b );
{5}      Writeln('Despues: ', a:4 , b:4 , c:4 );
    End.

```

que produce la siguiente salida:

```

    Antes :      3      5      7
Durante:      6     10      7
Durante:     12     10      7
Despues:     12      5      7

```

La variable *c* la conoce el procedimiento **Escribe** porque está declarada como variable global para todo el programa. La variable *y* se pasa por contenido y por tanto su multiplicación por 2 en el procedimiento no altera el valor del dato almacenado en la variable global *b*. Sin embargo, el argumento *x* se pasa por referencia y por tanto cada vez que se llama al procedimiento se multiplica por 2 el valor almacenado en la variable global *a*. Cuando una variable se pasa por referencia el compilador ha de interpretar que cada vez que se asigne un nuevo valor a esa variable se está almacenando un nuevo valor en el contenido de la dirección de memoria que corresponde a esa variable. Por eso se dice que se está transmitiendo al procedimiento la dirección de la variable.

Comprendiendo lo anterior está claro que las constantes no pueden ser transmitidas como argumentos pasados por referencia: las constantes están en posiciones en las que no se puede escribir durante la ejecución del programa. Por ejemplo, si el procedimiento **Escribe** se hubiera definido

```

      Procedure Escribe ( Var texto  : String;
                          Var x      : Integer;
                          y          : Integer );

```

el compilador avisaría de que en la instrucción **{3}** se espera como primer argumento del procedimiento una variable.

8.6 Definición diferida

El compilador transforma las ordenes *PASCAL* a lenguaje máquina empezando por el principio del programa y acabando por el **End**. Por tanto, cualquier procedimiento o función ha de declararse antes de ser utilizado para que el compilador cuando llegue al identificador correspondiente sepa interpretar qué instrucciones de la Unidad Central de Proceso debe generar. En general esto implica un orden mínimo a la hora de escribir el programa y llevar cuidado de no definir módulos en líneas posteriores a las que los utilizan. Sin embargo, puede haber ocasiones en las que se requiera que un módulo llame a otro módulo y que a su vez llame al primero. En este caso sería imposible definir los dos antes de referirlos y por tanto *PASCAL* proporciona la palabra reservada **FORWARD** para seguir al nombre y argumentos de un procedimiento o función cuyo contenido se especificará en líneas posteriores. En el programa **Dos** se ilustra el uso de **FORWARD** :

```

Program Dos;

```

```

Procedure B ( C : Char); FORWARD;

```

```

Procedure A ( c : Char);
Begin { A }
  If c < 'Z' Then B (c);
  Write ( c )
End; { A }

```

```

Procedure B ( c : Char);
Begin { B }
  A ( Succ (c) )
End; { B }

```

```

Begin { Dos }
  A ( 'A' );
End. { Dos }

```

El procedimiento **B** es necesario para definir el contenido de **A**, y a su vez **A** es necesario para definir el algoritmo siniestro que se explicita en **B**. El círculo vicioso se rompe adelantando al compilador que **B** es un **Procedure** que toma como argumento un dato **Char**. Cuando el compilador luego analiza el procedimiento **A** reconoce el identificador **B** y el hecho de que tome un argumento **Char**. Entonces, podrá generar código de la UCP en el que redireccione el cálculo a la dirección de memoria que ya tiene reservada para el módulo **B**. Posteriormente, cuando analice la especificación de los pasos a realizar en el módulo **B** generará el código UCP que constituirá el cálculo que se ha de realizar en el procedimiento **B**. Si todo esto parece enrevesado más lo es el modo en que se ha programado el algoritmo que realiza el programa **Dos**. Además, carece de ningún comentario para que se pueda calificar como un claro ejemplo de mala programación. El alumno, realizando las instrucciones del programa a mano tardará algún minuto en conocer cuál es la salida del programa.

8.7 Módulos y submódulos

Al igual que para un programa se pueden definir módulos, también para un módulo se pueden escribir otros módulos. Es decir declaraciones de procedimientos cuyo significado sólo conoce el módulo en el que se definen. Por ejemplo en el programa **Tres** la función **Final** es local al procedimiento **Eco**.

```

Program Tres;

```

```

  Procedure Eco;
  Var
    s : String;

  Function Final : Boolean;
  Var
    c : Char;
  Begin { Final }

```

```

        Writeln(' Acabo de hacer el ecoooooo ? (s/n) ');
        Readln(c);
        If
            (c = 's') Or (c = 'S')
        Then
            Final := True
        Else
            Final := False
        End; { Final }
    Begin { Eco }
        Repeat
            Readln(s);
            Writeln(s);
        Until Final;
    End; { Eco }

Begin { Tres }
    {1}
    {2}    Eco;
End. { Tres }

```

de modo que si en la línea etiquetada 1 del cuerpo del programa principal se añade la instrucción:

```

        If Final Exit;

```

para que el usuario pueda acabar tras la primera pregunta, el compilador avisará de que el identificador **Final** no está definido. Esta función está definida localmente para el procedimiento **Eco** exactamente como también es local a él la variable **s**. Esta posibilidad de anidamiento en la definición de módulos no se debe utilizar salvo en raras ocasiones pues es origen de muchas confusiones y es mejor optar por un estilo homogéneo en el que sólo se definen módulos para el programa principal

Un error muy común en la definición del contenido de las funciones es el olvidar asignar un valor al identificador de la función. Generalmente no se trata de una negligencia sintáctica sino del olvido de una posibilidad cuando se hacen asignaciones condicionales. Pero los efectos de este error son menos alarmantes que los de utilizar el identificador de la función en la parte derecha de una asignación en el cuerpo de la función. En ese caso, lo más probable es